Computer Vision Homework 2

葉騏緯
ID: 114598085

1. Median Filter: decide the kernel size, stride size, padding size by yourself. Find the best setting with the highest PSNR value that you can get.

Algorithm

We consider a median-based local operator with kernel radius r, padding (p_x, p_y), and stride (s_x, s_y) on a grayscale image. Let $\tilde{f}$ denote the padded extension of the input image f under the `reflect_101` boundary rule, namely mirror reflection without repeating the boundary sample. The output domain is

$$0 \le u < W_{\text{out}}, \quad 0 \le v < H_{\text{out}},$$

where

$$W_{\text{out}} = \left\lfloor \frac{W + 2p_x - (2r+1)}{s_x} \right\rfloor + 1, \quad H_{\text{out}} = \left\lfloor \frac{H + 2p_y - (2r+1)}{s_y} \right\rfloor + 1.$$

Then for each output coordinate (u, v), the program collects a $(2r+1) \times (2r+1)$ neighborhood from the padded image and defines

$$W(u, v) = \left(\tilde{f}(vs_y + j - p_y, us_x + i - p_x)\right)_{0 \le i, j \le 2r}.$$

Let the sorted samples of $W(u, v)$ be

$$s_0 \le s_1 \le \dots \le s_{n-1},$$

where

$$n = (2r+1)^2.$$

Then the median filter output is defined by

$$g(u, v) = s_{\lfloor \frac{n}{2} \rfloor}.$$

Therefore, when stride is larger than 1, the operator is not producing one output pixel for every original input location; instead, it evaluates only the sampled neighborhoods determined by the stride.

Unlike linear averaging filters, the median filter is robust to outliers: if fewer than half of the samples in the window are corrupted by extreme values, the output is unaffected.

When stride is greater than 1, the output size changes, so PSNR is computed against an aligned reference image g_{ref} with the same output geometry rather than against the original full-resolution image:

$$\text{PSNR}(g, g_{\text{ref}}).$$

Implementation

To compute the median efficiently on an 8-bit grayscale image, the program uses a histogram-based method. Since each pixel value lies in $\{0, 1, \dots, 255\}$, it is enough to count how many samples fall into each of the 256 bins and then scan the cumulative count until the middle position is reached. This avoids fully sorting the window values and matches the median definition exactly.

For Q1, the tested parameter range is

$$r \in \{1,2,3,4,5\}, \quad p_x = p_y \in \{0,1,2,3,4,5\}, \quad s_x = s_y \in \{1,2,3\}.$$

According to `CKS_Q1_psnr.csv`, the best configuration is

$$r = 1, \quad k = 3, \quad p_x = p_y = 5, \quad s_x = s_y = 3,$$

with

$$\mathrm{PSNR} = 26.26 \text{ dB}.$$

Figure 1: Median filter result for Q1. The best configuration selected from `CKS_Q1_psnr.csv` is radius $r = 1$ (kernel 3×3), padding $p_x = p_y = 5$, stride $s_x = s_y = 3$, with PSNR 26.26 dB.

2. Gaussian Filter: decide the kernel size, stride size, padding size by yourself. Find the best setting with the highest PSNR value that you can get

Algorithm

We consider a Gaussian smoothing operator with kernel radius r, padding (p_x, p_y), and stride (s_x, s_y) on a grayscale image. Let $\tilde{f}$ denote the padded extension of the input image f under the `reflect_101` boundary rule. The output domain is

$$0 \le u < W_{\text{out}}, \quad 0 \le v < H_{\text{out}},$$

where

$$W_{\text{out}} = \left\lfloor \frac{W + 2p_x - (2r+1)}{s_x} \right\rfloor + 1, \quad H_{\text{out}} = \left\lfloor \frac{H + 2p_y - (2r+1)}{s_y} \right\rfloor + 1.$$

Let the one-dimensional Gaussian kernel be

$$h(t) = \frac{\exp\left(-\frac{t^2}{2\sigma^2}\right)}{\sum_{k=-r}^{r}} \exp\left(-\frac{k^2}{2\sigma^2}\right), \quad -r \le t \le r,$$

where $\sigma > 0$ is the Gaussian standard deviation. Then the filtered output is defined by the separable form

$$g(u,v) = \sum_{j=0}^{2r} \sum_{i=0}^{2r} h(i-r)h(j-r)\tilde{f}\big(vs_y + j - p_y, us_x + i - p_x\big).$$

The Gaussian filter is a linear weighted average. Nearby pixels contribute more strongly than distant pixels because the weights decay with distance from the kernel center. As a result, the filter suppresses high-frequency noise, but it also smooths edges and fine details.

As in Q1, when stride is greater than 1, PSNR is computed against an aligned reference image g_{ref} with the same output geometry:

$$\text{PSNR}(g, g_{\text{ref}}).$$

Implementation

The implementation uses the default Gaussian standard deviation

$$\sigma = \frac{r}{2} + 0.5,$$

which is the value generated by `default_gaussian_sigma<Radius>()` in `filter.hpp`.

Instead of applying a full two-dimensional convolution directly, the program uses the separability of the Gaussian kernel. It first performs a horizontal 1D Gaussian pass into a temporary buffer, and then applies a vertical 1D Gaussian pass to produce the final output. This reduces the computation cost from a full $(2r+1) \times (2r+1)$ weighted sum per pixel to two one-dimensional passes while preserving the same Gaussian result.

For Q2, the tested parameter range is

$$r \in \{1,2,3,4,5\}, \quad p_x = p_y \in \{0,1,2,3,4,5\}, \quad s_x = s_y \in \{1,2,3\}.$$

According to `CKS_Q2_psnr.csv`, the best configuration is

$$r = 3, \quad k = 7, \quad p_x = p_y = 5, \quad s_x = s_y = 3,$$

with

$$\text{PSNR} = 21.88 \text{ dB}.$$

Figure 2: Gaussian filter result for Q2. The best configuration selected from `CKS_Q2_psnr.csv` is radius $r = 3$ (kernel 7×7), padding $p_x = p_y = 5$, stride $s_x = s_y = 3$, with PSNR 21.88 dB.

3. Combine: by combining median and gaussian filter, find the best image results with the highest PSNR value that you can get

Algorithm

We consider a two-stage filtering operator obtained by composing a median filter and a Gaussian filter. Let

$$M_{r_m,p_m}$$

denote the median operator with radius r_m and padding p_m, and let

$$G_{r_g,p_g}$$

denote the Gaussian operator with radius r_g and padding p_g. In Q3, both stages use stride 1, and the Gaussian standard deviation σ follows the same default rule as in Q2:

$$\sigma = \frac{r_g}{2} + 0.5.$$

Two orders are considered:

$$g = G_{r_g,p_g}\big(M_{r_m,p_m}(f)\big)$$

and

$$g = M_{r_m,p_m}\big(G_{r_g,p_g}(f)\big).$$

The noisy input in this task contains corruption that can be interpreted as a mixture of impulse-like outliers and broader fluctuations. Under this assumption, `median -> gaussian` is preferable: the median stage first removes the sparse outliers, and the Gaussian stage then smooths the remaining fluctuations. In the opposite order, the Gaussian stage first diffuses impulse energy into neighboring pixels, reducing the outlier amplitude but spreading its effect; the subsequent median stage then operates on corruption that is no longer sparse, so its outlier-rejection property is weakened. This explains why the benchmark favors `median -> gaussian`.

Implementation

In this benchmark, the searched parameter space is

$$r_m \in \{1,2,3,4,5\}, \quad p_m \in \{0,1,2,3,4,5\},$$

$$r_g \in \{1,2,3,4,5\}, \quad p_g \in \{0,1,2,3,4,5\},$$

together with both filter orders. According to `CKS_Q3_psnr.csv` and `report_summary.log`, the best configuration is

$$g = G_{1,5}\big(M_{1,5}(f)\big),$$

that is, `median -> gaussian` with median radius 1, median padding 5, Gaussian radius 1, and Gaussian padding 5, with

$$\mathrm{PSNR} = 26.52 \text{ dB}.$$

Figure 3: Combined filtering result for Q3. The best configuration selected from `CKS_Q3_psnr.csv` is `median -> gaussian` with median radius 1 (kernel 3×3), median padding 5, Gaussian radius 1 (kernel 3×3), Gaussian padding 5, and PSNR 26.52 dB.

4. Image Histogram: accumulate the number of each pixel value, then draw a histogram for each output image from previous tasks.

Algorithm

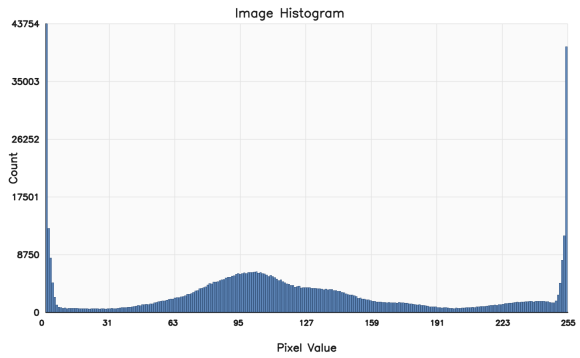
For a grayscale image f, the histogram is the discrete count function

$$H(k) = |\{(x, y) : f(x, y) = k\}|, \quad k \in \{0, 1, ..., 255\}.$$

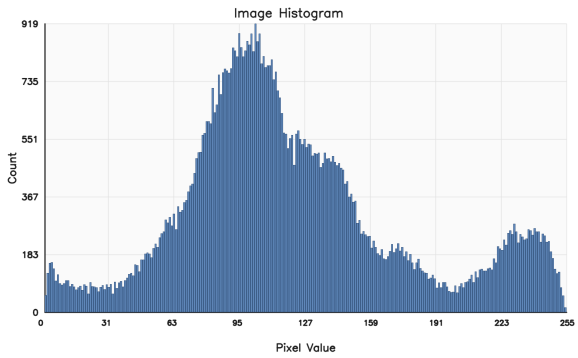
That is, $H(k)$ counts how many pixels in the image have intensity value k. The histogram therefore summarizes the intensity distribution of the image without using spatial position information.

Implementation

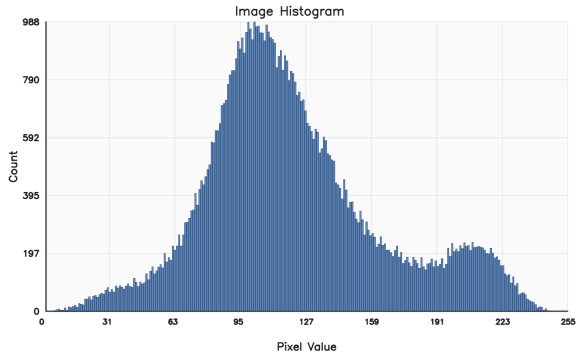
The program accumulates the histogram by scanning all pixels once and incrementing one of 256 bins according to the grayscale value. After accumulation, the counts are rendered as a bar chart image with labeled axes, grid lines, and one bar for each intensity level from 0 to 255.



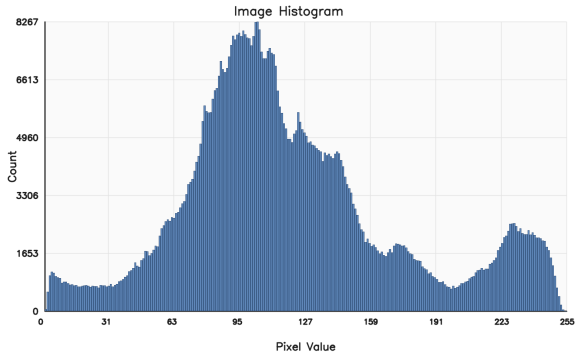
(a) Noisy input histogram.



(b) Q1 median-filter histogram.



(c) Q2 Gaussian-filter histogram.



(d) Q3 combined-filter histogram.

Figure 4: Histograms of the noisy image and the filtered outputs. (a) Histogram of the noisy input image. (b) Histogram after median filtering with the best Q1 configuration. (c) Histogram after Gaussian filtering with the best Q2 configuration. (d) Histogram after the best combined filter from Q3.

Comparing Fig. 4(a)-4(d), the noisy image in Fig. 4(a) shows sharp spikes near intensities 0 and 255, which are clear signatures of impulse-like corruption. After median filtering in Fig. 4(b), these extreme spikes are largely removed and the histogram returns to a clearer bimodal structure, corresponding more closely to the dark and bright regions of the scene. In Fig. 4(c), Gaussian filtering smooths the histogram itself: neighboring bins vary more continuously, but the main peaks are lower because the weighted average spreads intensities across nearby values. Finally, Fig. 4(d) combines these two effects: the extreme spikes are suppressed as in Q1, while the overall distribution is smoother than in Fig. 4(b). This is consistent with the fact that Q3 achieves the best PSNR among the tested outputs.